فصل ۳: کاهش ابعاد

مقدمه

در این فصل، تعداد ویژگی‌های مدل‌ها در یادگیری ماشین را کاهش خواهیم داد. این یک عملیات بسیار مهم است زیرا بعضی اوقات مجموعه‌های داده ستون ورودی زیادی دارند و کاهش تعداد ستون‌ها، مدل‌های ساده‌تری را ایجاد می‌کند که برای پیش‌بینی محاسبات کمتری لازم دارند. مدل اصلی مورد استفاده در این بخش، تجزیه و تحلیل مولفه‌های اصلی (PCA) است. به لطف واریانس توضیح داده شده، در PCA، لازم نیست بدانید که می‌توانید مجموعه داده را به چند ویژگی کاهش دهید. یک مدل مشابه در عملکرد، singular value decomposition است.

کاهش ابعاد با PCA

PCA اولین روش پیشرفته‌ای است که در این کتاب مورد بحث قرار گرفته است. در حالی که همه موارد که تاکنون بحث شد آماری و ساده بودند، PCA با ترکیب آماری و جبر خطی یک مرحله پیش پردازش تولید می‌کند که می‌تواند به کاهش ابعاد کمک کند. PCA در ماژول scikit-learn نیز قرار دارد. چندین روش دیگر نیز وجود دارد که بعداً توضیح داده خواهد شد.

به مثال زیر توجه کنید

```
from sklearn import datasets
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
%matplotlib inline
iris = datasets.load_iris()
iris_X = iris.data
y = iris.target
```

ماژول decomposition را وارد کنید و شی PCA را مقداردهی اولیه نمایید

```
from sklearn import decomposition
pca = decomposition.PCA(copy=True, iterated_power='auto',
n_components=None,
random_state=None,svd_solver='auto', tol=0.0, whiten=False)
```

اکنون که شی PCA (نمونه PCA) ایجاد شده است، با فراخوانی fit_transformmetode داده‌ها با iris_X را به سادگی تبدیل می‌کنیم.

```
iris_pca = pca.fit_transform(iris_X)
iris_pca[:5]
output: array([[-2.68412563e+00,  3.19397247e-01, -2.79148276e-02,
        -2.26243707e-03],
       [-2.71414169e+00, -1.77001225e-01, -2.10464272e-01,
        -9.90265503e-02],
       [-2.88899057e+00, -1.44949426e-01,  1.79002563e-02,
        -1.99683897e-02],
       [-2.74534286e+00, -3.18298979e-01,  3.15593736e-02,
         7.55758166e-02],
       [-2.72871654e+00,  3.26754513e-01,  9.00792406e-02,
         6.12585926e-02]])
```

برای نمایش توضیحات واریانس از دستور زیر استفاده می‌کنیم

```
pca.explained_variance_ratio_
output:
array([0.92461872, 0.05306648, 0.01710261, 0.00521218])
```

PCA دارای یک تعریف عمومی ریاضی و یک مورد استفاده خاص در تجزیه و تحلیل داده‌ها است. PCA مجموعه جهات متعامدی را پیدا می‌کند که ماتریس داده اصلی را نشان می‌دهد. به طور کلی، PCA مجموعه داده اصلی را در فضای جدیدی نگاشت می‌کند که هر یک از بردارهای ستون جدید ماتریس به صورت متعامد هستند. از دیدگاه تجزیه و تحلیل داده‌ها، PCA ماتریس کوواریانس داده‌ها را به بردارهای ستونی تبدیل می‌کند که می‌تواند

درصدهای خاصی از واریانس را توضیح دهد. به عنوان مثال، با مجموعه داده‌های iris، ۹۲.۵ درصد از واریانس مجموعه داده‌های کلی را می‌توان با اولین مولفه توضیح داد.

اغلب اوقات، الگوریتم‌های اعمال شده در مجموعه داده‌های با ابعاد بالا، در آموزش اولیه باعث بیش برازش (over-fitting) می‌شوند و بنابراین عمومیت مجموعه آزمون (test) را از دست می‌دهند. اگر بیشتر ساختار اساسی داده‌ها بتوانند با ابعاد کمتری نشان داده شوند، به عنوان تعادل ارزشمند محسوب می‌شود.

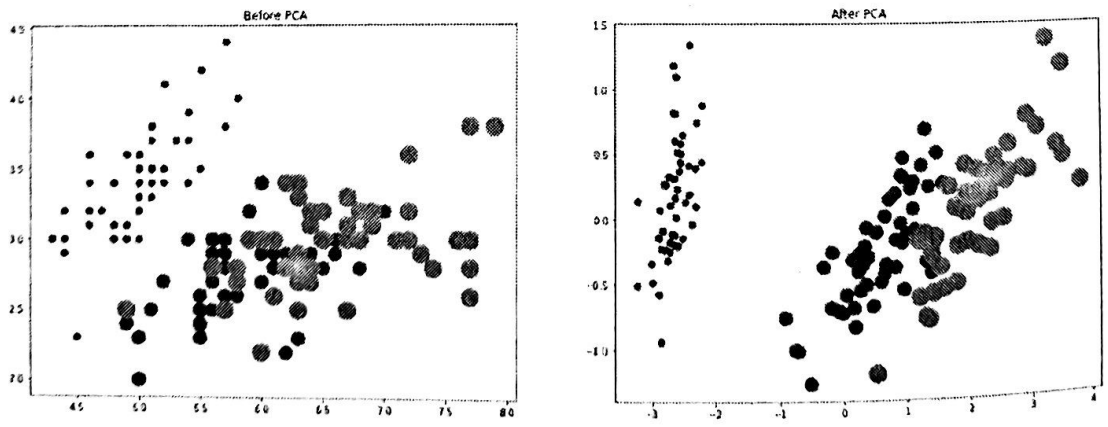
```
pca = decomposition.PCA(n_components=2)
iris_X_prime = pca.fit_transform(iris_X)
iris_X_prime.shape
output: (150, 2)
```

ابعاد ماتریس داده اکنون از ۴×۱۵۰ به صورت ۲×۱۵۰ تغییر کرده است. قابلیت تفکیک کلاس‌ها حتی پس از کاهش ابعاد به دو بعد قابل مشاهده است. می‌توان مشاهده کرد که چه مقدار از واریانس توسط دو جز components نشان داده شده است.

```
pca.explained_variance_ratio_.sum()
output: 0.977685206318795
```

برای تجسم آنچه PCA انجام داده است، بیایید دو بعد اول مجموعه داده iris را با تصاویر قبل از تغییر شکل PCA ترسیم کنیم (شکل ۳-۱).

```
fig = plt.figure(figsize=(20,7))
ax = fig.add_subplot(121)
ax.scatter(iris_X[:,0],iris_X[:,1],c=y,s=40)
ax.set_title('Before PCA')
ax2 = fig.add_subplot(122)
ax2.scatter(iris_X_prime[:,0],iris_X_prime[:,1],c=y,s=40)
ax2.set_title('After PCA')
```



شکل (۳-۱): خروجی PCA

PCA را می‌توان با مقدار واریانس توضیح داده شده از همان ابتدا ایجاد کرد. به عنوان مثال، اگر بخواهیم حداقل ۹۸ درصد از واریانس را توضیح دهیم، PCA به شرح زیر ایجاد می‌شود:

```
pca = decomposition.PCA(n_components=.98)
iris_X_prime = pca.fit(iris_X).transform(iris_X)
pca.explained_variance_ratio_.sum()
output: 0.9947878161267247
```

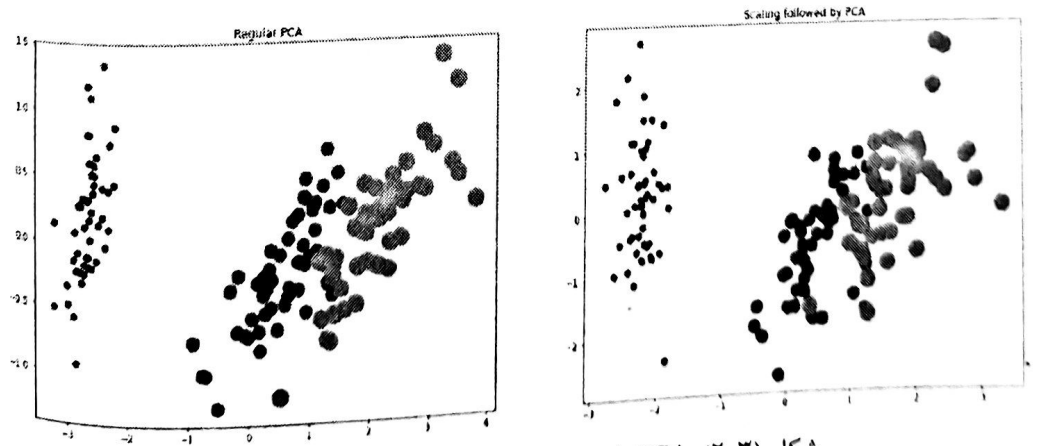
از آنجایی که ما می‌خواستیم واریانس را کمی بیشتر از مقدار قبلی (0.977685206318795) توضیح دهیم، مولفه سوم اضافه گردید.

توجه:

حتی اگر ابعاد نهایی داده‌ها دو یا سه باشد، این دو یا سه ستون حاوی اطلاعاتی از هر چهار ستون اصلی هستند.

توصیه می‌شود قبل از استفاده از PCA مقیاس‌بندی (scale) را انجام دهید. این کار به صورت زیر امکان‌پذیر است

```
from sklearn import preprocessing
iris_X_scaled = preprocessing.scale(iris_X)
pca = decomposition.PCA(n_components=2)
iris_X_scaled = pca.fit_transform(iris_X_scaled)
fig = plt.figure(figsize=(20,7))
ax = fig.add_subplot(121)
ax.scatter(iris_X_prime[:,0],iris_X_prime[:,1],c=y,s=40)
ax.set_title('Regular PCA')
ax2 = fig.add_subplot(122)
ax2.scatter(iris_X_scaled[:,0],iris_X_scaled[:,1],c=y,s=40)
ax2.set_title('Scaling followed by PCA')
```



شکل (۳-۲): PCA با خروجی مقیاس شده- PCA معمولی

می‌توانید با کمک خط لوله به صورت زیر مقیاس‌بندی را انجام دهیم

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
pipe = Pipeline([('scaler', StandardScaler()),
('pca',decomposition.PCA(n_components=2))])
iris_X_scaled = pipe.fit_transform(iris_X)
```

توجه:

استفاده از خطوط لوله باعث کاهش خطا و اشکال‌زدایی کدهای پیچیده می‌گردد.

استفاده از فاکتور آنالیز برای تجزیه (Factor analysis)

فاکتور آنالیز (Factor analysis) یکی دیگر از فن‌آوری‌هایی است که می‌توانیم از آن برای کاهش ابعاد استفاده کنیم. با این حال، Factor analysis برخلاف PCA فرضیاتی را ارائه می‌دهد و فرض اصلی این است که ویژگی‌های ضمنی مسئول ویژگی‌های مجموعه داده هستند.

از آنجا که فاکتور آنالیز یک تغییر احتمالی است، ما می‌توانیم جنبه‌های مختلفی مانند درستنمایی (log likelihood) مشاهدات تحت مدل را بررسی کنیم و بهتر است، درستنمایی را در بین مدل‌ها مقایسه کنیم. فاکتور آنالیز خالی از اشکال نیست. دلیل این امر این است که شما مدل را برای پیش‌بینی خروجی برازش نکرده‌اید، بلکه شما به عنوان یک مرحله آماده‌سازی، مدل را برازش (fit) می‌کنید. در کل این عمل کار می‌کند، ولی هنگام آموزش مدل واقعی، خطاها بیشتر می‌شوند.

فاکتور آنالیز مشابه PCA است، با این حال، یک تمایز مهم وجود دارد. PCA یک تبدیل شکل خطی داده‌ها به فضای متفاوتی است که در آن اولین مولفه، واریانس داده‌ها را توضیح می‌دهد و مولفه‌های بعدی بر مولفه اول عمود هستند. به عنوان مثال، می‌توانید PCA را تصور کنید که یک مجموعه داده از N بعد را می‌گیرد و به یک فضای M بعدی نگاشت می‌کند، $M < N$.

از طرف دیگر، فاکتور آنالیز با این فرض وجود دارد که فقط M ویژگی مهم وجود دارد و یک ترکیب خطی از این ویژگی‌ها (به علاوه نویز) مجموعه داده را در N بعد ایجاد می‌کند. به بیان دیگر، شما روی متغیر خروجی رگرسیون انجام نمی‌دهید، بلکه برای تعیین فاکتورها، رگرسیون را روی ویژگی‌ها انجام می‌دهید. علاوه بر این، یک اشکال بزرگ این

است که شما نمی‌دانید به چه تعداد ستون می‌توانید داده‌ها را کاهش دهید. PCA با توضیح معیار واریانس شما را در این راهنمایی می‌کند.

استفاده از هسته PCA برای کاهش بعد غیرخطی

بیشتر تکنیک‌های آماری به طور ذاتی خطی هستند، بنابراین برای غیرخطی بودن، ممکن است نیاز به اعمال برخی تحولات داشته باشیم. البته PCA یک تبدیل خطی است. در این دستورالعمل، ما به دنبال استفاده از تغییرات غیرخطی هستیم و سپس PCA را برای کاهش ابعاد اعمال می‌کنیم.

اگر داده‌ها همیشه به صورت خطی قابل تفکیک باشند زندگی بسیار آسان خواهد بود، اما متأسفانه اینگونه نیست. هسته PCA می‌تواند به دور زدن این مسئله کمک کند. داده‌ها ابتدا از طریق تابع هسته اجرا می‌شوند که داده‌ها را بر روی فضای دیگری نگاشت می‌کند؛ و سپس PCA اجرا می‌شود.

برای آشنایی با توابع هسته، تمرین خوبی خواهد بود که به چگونگی تولید داده‌هایی که توسط توابع هسته موجود در هسته PCA قابل تفکیک هستند، فکر کنید. در اینجا، ما این کار را با هسته کسینوس انجام می‌دهیم. این دستورالعمل کمی بیشتر از دستور العمل‌های قبلی تئوری خواهد بود.

قبل از شروع، مجموعه داده iris را بارگیری کنید

```
from sklearn import datasets, decomposition
iris = datasets.load_iris()
iris_X = iris.data
```

هسته کسینوس با مقایسه زاویه بین دو نمونه نشان داده شده در فضای ویژگی کار می‌کند. برای یادآوری، کسینوس بین دو بردار با فرمول زیر داده می‌شود

$$Cos(\theta) = \frac{A.B}{||A||\ ||B||}$$

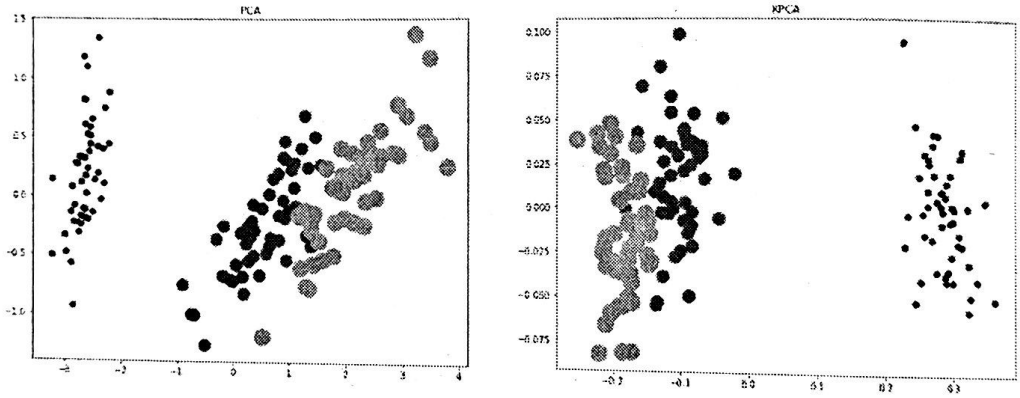
این بدان معنی است که کسینوس بین A و B ضرب داخلی دو بردار است که نرمال می‌شود.

نکته:

اندازه بردار به صورت زیر محاسبه می‌شود:

$$||A|| = \left(\sum_{i=1}^{N} |A_i|^2\right)^{\frac{1}{2}} = A_1^2 + A_2^2 + \cdots + A_N^2$$

```
pca = decomposition.PCA(n_components=2(
iris_X_prime = pca.fit_transform(iris_X(
kpca = decomposition.KernelPCA(kernel='cosine', n_components=2(
iris_X_primekpca = kpca.fit_transform(iris_X)
fig = plt.figure(figsize=(20,7))
ax = fig.add_subplot(121)
ax.scatter(iris_X_prime[:,0],iris_X_prime[:,1],c=y,s=40)
ax.set_title(' PCA')
ax2 = fig.add_subplot(122)
ax2.scatter(iris_X_primekpca[:,0],iris_X_primekpca[:,1],c=y,s=40)
ax2.set_title('KPCA')
```



شکل (۳-۳): KPCA در مقایسه با PCA معمولی

علاوه بر هسته کسینوس، چندین هسته مختلف وجود دارد. حتی می‌توانید عملکرد هسته خود را بنویسید. هسته‌های موجود به شرح زیر است:

- Poly (polynomial)
- RBF (radial basis function)
- Sigmoid
- Cosine
- Pre-computed

همچنین گزینه‌هایی وجود دارد که به انتخاب هسته بستگی دارد. به عنوان مثال، آرگومان درجه برای هسته، poly، RBF و سیگموئید. (گاما بر RBF یا چند هسته‌ای تأثیر می‌گذارد.) تابع SVM عملکرد هسته RBF را با جزئیات بیشتری پوشش می‌دهد.

توجه:

روش‌های هسته برای تفکیک عالی هستند، اما اگر بدون توجه استفاده شوند، می‌توانند باعث بیش برازش نیز شوند. پس بهتر است اطمینان حاصل کنید که آن‌ها را به درستی آموزش یا تست می‌کنید.

خوشبختانه هسته‌های موجود عملکردهای روان، مداوم و قابل تفکیکی هستند. آن‌ها لبه‌های دندانه‌دار درختان رگرسیون را ایجاد نمی‌کنند.

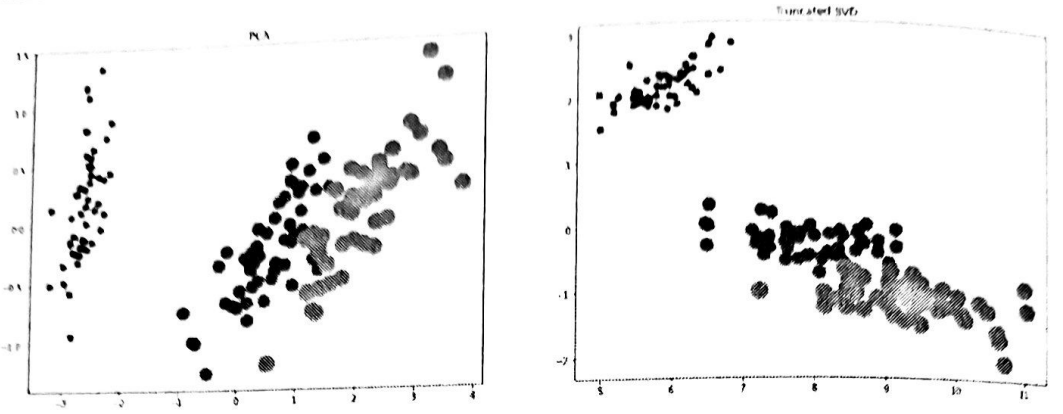
استفاده از SVD کوتاه شده برای کاهش ابعاد

SVD کوتاه یک تکنیک فاکتوراسیون ماتریسی است که ماتریس M را به سه ماتریس U، Σ و V فاکتور می‌دهد. این روش بسیار شبیه PCA است، با این تفاوت که SVD بر روی ماتریس داده اعمال می‌شود، در حالی‌که برای PCA، روی ماتریس کوواریانس اعمال می‌شود. به طور معمول، SVD برای پیدا کردن اجزای اصلی ماتریس استفاده می‌شود.

SVD کوتاه شده با SVD معمولی متفاوت است به این دلیل که در آن تعداد ستون‌ها برابر با ستون‌های قطع شده است. به عنوان مثال، با توجه به ماتریس n × n، SVD ماتریس‌هایی با n ستون تولید می‌کند، در حالی‌که SVD کوتاه شده ماتریس‌هایی با تعداد ستون مشخص تولید می‌کند.

به مثال زیر توجه کنید

```
from sklearn.datasets import load_iris
iris = load_iris()
iris_X = iris.data
y = iris.target
svd.explained_variance_ratio_
output: array([0.52875361, 0.44845576])
fig = plt.figure(figsize=(20,7))
ax = fig.add_subplot(121)
ax.scatter(iris_X_prime[:,0],iris_X_prime[:,1],c=y,s=40)
ax.set_title(' PCA')
ax2 = fig.add_subplot(122)
ax2.scatter(iris_transformed[:,0],iris_transformed[:,1],c=y,s=40)
ax2.set_title('Truncated SVD')
```



شکل (۳-۴): Truncated SVD در مقایسه با PCA معمولی

نتایج بسیار خوب به نظر می‌رسند. مانند PCA، واریانس را با دستور زیر نمایش داده می‌شود

```
svd.explained_variance_ratio_
```

اکنون که ما نحوه عملکرد scikit-learn را بررسی کردیم، بیایید ببینیم چگونه می‌توانیم فقط از SciPy استفاده کنیم و در این زمینه کمی اطلاعات خود را افزایش دهیم. ابتدا باید از SciPy linalg برای انجام SVD استفاده کنیم

```
from scipy.linalg import svd
import numpy as np
D = np.array([[1, 2], [1, 3], [1, 4]])
D
Output: array([[1, 2],
       [1, 3],
       [1, 4]])
U, S, V = svd(D, full_matrices=False)
U.shape, S.shape, V.shape
Output: ((3, 2), (2,), (2, 2))
np.dot(U.dot(np.diag(S)), V)
output: array([[1., 2.],
       [1., 3.],
       [1., 4.]])
new_S = S[0]
new_U = U[:, 0]
new_U.dot(new_S)
output: array([-2.20719466, -3.16170819, -4.11622173])
```

ما می‌توانیم ماتریس اصلی را به وسیله U، S و V بازسازی کنیم

```
np.dot(U.dot(np.diag(S)), V)
output: array([[1., 2.],
       [1., 3.],
       [1., 4.]])
```

ماتریسی که در واقع توسط SVD کوتاه شده برگردانده می‌شود، محصول ضرب داخلی از ماتریس‌های U و S است. اگر بخواهیم برش را شبیه‌سازی کنیم، کوچک‌ترین مقادیر منفرد و بردارهای ستون مربوطه را از U حذف خواهیم کرد. بنابراین، برای یافتن یک جز واحد، عمل زیر را انجام می‌دهیم

```
new_S = S[0]
new_U = U[:, 0]
new_U.dot(new_S)
output: array([-2.20719466, -3.16170819, -4.11622173])
```

به طور کلی، اگر بخواهیم به بعد t برسیم مقادیر منفرد N - t را حذف می‌کنیم.

Sign flipping

بسته به حالت مولد اعداد تصادفی، برازش شدن‌های پی‌درپی SVD کوتاه می‌توانند علامت (sign) خروجی را تغییر دهند. به منظور جلوگیری از این موارد، توصیه می‌شود SVD کوتاه شده را یک بار برازش کنید و از آن به بعد از تبدیل استفاده کنید. این دلیل خوبی دیگر برای استفاده از خطوط لوله (pipeline) است.

```
tsvd = TruncatedSVD(2)
tsvd.fit(iris_X)
tsvd.transform(iris_X)
```

ماتریس‌های پراکنده (sparse matrix)

یک مزیت SVD کوتاه شده نسبت به PCA این است که SVD کوتاه شده توانایی کار روی ماتریس‌های پراکنده را دارد، در حالی که PCA قادر نمی‌باشد. علت این است که PCA نیاز به محاسبه ماتریس کوواریانس در کل داده‌ها دارد که کار پر هیزینه‌ای می‌باشد.

استفاده از تجزیه برای طبقه‌بندی با فرهنگ لغت (Dictionary Learning)

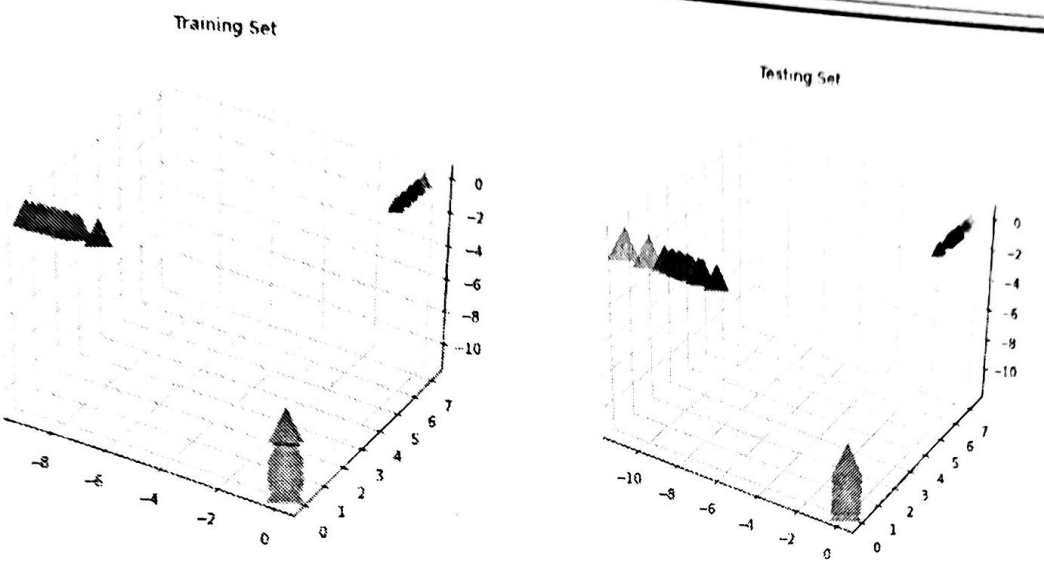
نشان خواهیم داد که چگونه می‌توان از یک روش تجزیه برای طبقه‌بندی استفاده کرد. یادگیری فرهنگ لغت (Dictionary Learning) تلاش می‌کند تا یک مجموعه داده را به نمایشی پراکنده تبدیل کند.

- مجموعه داده iris را بارگذاری می‌کنیم
- از سه مولفه برای نشان دادن سه گونه استفاده کنید
- هر داده را تبدیل می‌کنیم تا بتوانیم بعد از آموزش، طبقه‌بندی را روی نقاط حاصل آزمایش کنیم
- آن را تست می‌کنیم

```
from sklearn.datasets import load_iris
iris = load_iris()
iris_X = iris.data
y = iris.target
X_train = iris_X[::2]
X_test = iris_X[1::2]
y_train = y[::2]
y_test = y[1::2]
from sklearn.decomposition import DictionaryLearning
dl = DictionaryLearning(3)
transformed = dl.fit_transform(X_train)
transformed[:5]
output: array([[0.       , 6.344753  , 0.       ],
       [0.       , 5.83576099, 0.       ],
       [0.       , 6.32037564, 0.       ],
       [0.       , 5.89324204, 0.       ],
       [0.       , 5.4522191 , 0.       ]])
test_transform = dl.transform(X_test)
```

حال آن را نمایش می‌دهیم

```
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(14,7))
ax = fig.add_subplot(121, projection='3d')
ax.scatter(transformed[:,0],transformed[:,1],transformed[:,2],c=y_train,marker = '^')
ax.set_title("Training Set")
ax2 = fig.add_subplot(122, projection='3d')
ax2.scatter(test_transform[:,0],test_transform[:,1],test_transform[:,2],c=y_test,
marker = '^')
ax2.set_title("Testing Set")
```



شکل (۳-۵): نمایش داده‌ها در فضای ۳ بعدی

اگر خوب دقت کنید، می‌بینید که خطای آموزش وجود دارد. یکی از کلاس‌ها به اشتباه طبقه‌بندی شده است. با توجه به نمودارهایی که در مثال‌های قبلی رسم کردیم، کلاس‌های قرمز و سبز دو طبقه‌ای بودند که اغلب نزدیک هم ظاهر می‌شدند.

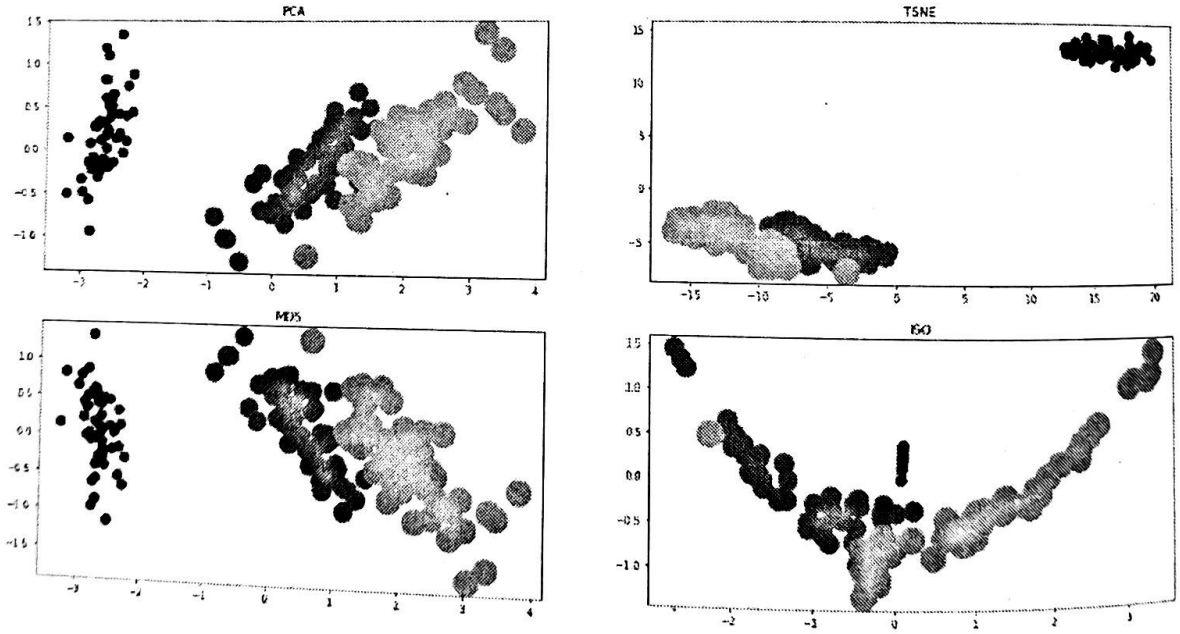
DictionaryLearning دارای پس‌زمینه پردازش سیگنال و عصب‌شناسی است. ایده این است که فقط تعداد کمی از ویژگی‌ها می‌توانند در هر زمان فعال باشند. بنابراین، Dictionary Learning تلاش می‌کند تا با توجه به صفر بودن بیشتر ویژگی‌ها، نمایشی مناسب از داده‌های اساسی ارائه کند.

انجام کاهش ابعاد با t-SNE

مجموعه داده iris را بارگیری کنید
ماژول manifold را فراخوانی می‌کنیم
تمام تبدیلات را روی داده‌های اعمال کنید. یکی از آن‌ها t-SNE است
آنها را نمایش می‌دهیم

```
from sklearn.datasets import load_iris
iris = load_iris()
iris_X = iris.data
y = iris.target
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE, MDS, Isomap
#Load visualization library
import matplotlib.pyplot as plt
```

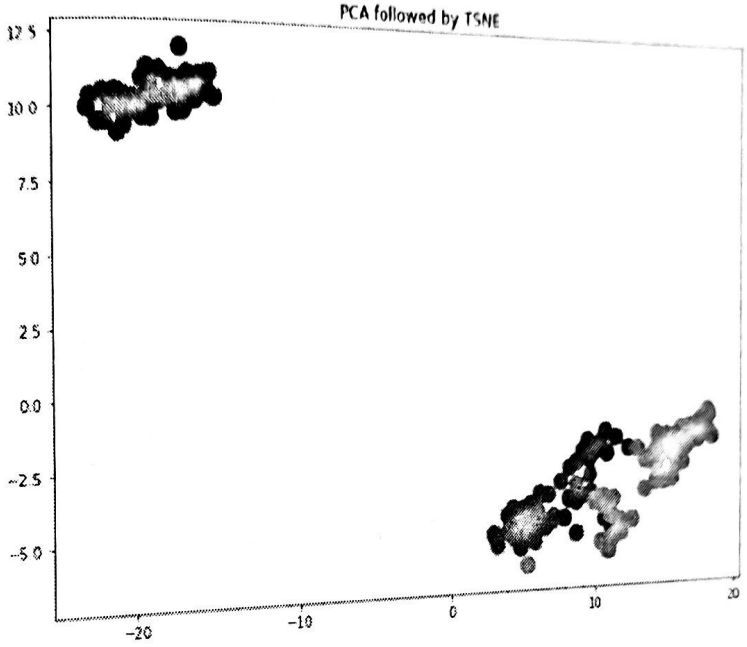
```
%matplotlib inline
iris_pca = PCA(n_components = 2).fit_transform(iris_X)
iris_tsne = TSNE(learning_rate=200).fit_transform(iris_X)
iris_MDS = MDS(n_components = 2).fit_transform(iris_X)
iris_ISO = Isomap(n_components = 2).fit_transform(iris_X)
plt.figure(figsize=(20, 10))
plt.subplot(221)
plt.title('PCA')
plt.scatter(iris_pca [:, 0], iris_pca [:, 1], c=y)
plt.subplot(222)
plt.scatter(iris_tsne[:, 0], iris_tsne[:, 1], c=y)
plt.title('TSNE')
plt.subplot(223)
plt.scatter(iris_MDS[:, 0], iris_MDS[:, 1], c=y)
plt.title('MDS')
plt.subplot(224)
plt.scatter(iris_ISO[:, 0], iris_ISO[:, 1], c=y)
plt.title('ISO')
```



شکل (۳-۶): مقایسه pca,tsne,MDS, ISO

الگوریتم t-SNE اخیراً محبوب شده است، با این وجود زمان و قدرت زیادی برای محاسبات لازم دارد. علاوه بر این، در مواردی که ابعاد داده‌ها بسیار زیاد است (بیش از 50 ستون)، پیشنهاد می‌شودکه PCA یا SVD کوتاه قبل از t-SNE را انجام دهید.

```
iris_pca_then_tsne = TSNE(learning_rate=200).fit_transform(iris_pca)
plt.figure(figsize=(10, 7))
plt.scatter(iris_pca_then_tsne[:, 0], iris_pca_then_tsne[:, 1], c=y)
plt.title("PCA followed by TSNE")
```

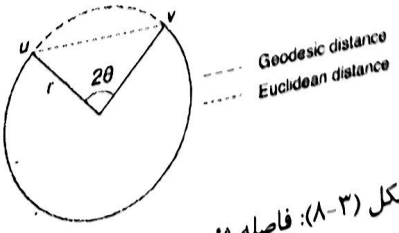


شکل (۳-۷): نمایش tSNE

مدل manifold در یادگیری فضاهای دارای ابعاد بالا در دو یا سه بعد مفید است. الگوریتم‌های آن اطلاعات مربوط به ساختار محلی اطراف یک نقطه را جمع و به دنبال حفظ آن هستند. به عنوان مثال، الگوریتم Isomap تلاش می‌کند تا فاصله ژئودزیکی بین همه نقاط یک الگوریتم را حفظ کند، نزدیک‌ترین همسایه را جستجو می‌کند. هدف این الگوریتم حفظ فاصله‌ها و ساختار هندسی محلی یک منیفولد است. الگوریتم مقیاس‌گذاری چند بعدی (multi-dimensional scaling MDS) نیز فواصل را در نظر می‌گیرد.

نکته:

تفاوت فاصله ژئودیزیک و فاصله اقلیدسی در شکل زیر نشان داده شده است



شکل (۳-۸): فاصله ژئودیزیکی و اقلیدسی

t-SNE فاصله اقلیدسی بین جفت نقاط مجموعه داده را به احتمالات تبدیل می‌کند. در اطراف هر نقطه یک گاوسی در آن نقطه در نظر می‌گیرد و توزیع احتمال نشان دهنده شانس همسایگی با نقاط دیگر است. نقاط بسیار دور از یکدیگر شانس کمتری برای همسایگی دارند. در اینجا، ما مکان‌های نقطه را به فاصله و سپس احتمالات تبدیل کرده‌ایم. t-SNE با استفاده از احتمال همسایگی دو نقطه ساختار محلی را به خوبی حفظ می‌کند.

به طور کلی، روش‌های منیفولد با بررسی همسایگان هر نقطه، که نمایانگر ساختار محلی یک منیفولد هستند، شروع و سعی در حفظ آن ساختار محلی به روش‌های مختلف دارند. این مانند این است که شما در اطراف محله خود قدم بزنید و از نقشه دو بعدی محل اطراف خود استفاده کنید و به ساختار سه بعدی توجهی نکنید.

کاهش ابعاد با کمک خطوط لوله

در اینجا خواهیم دید که تخمین زننده‌های مختلف متشکل از کاهش ابعاد و ماشین بردار پشتیبان چگونه عمل می‌کنند.

یک لوله را با دو قسمت اصلی تهیه کنید:

- داده‌های iris را بارگذاری کنید
- تخمین زننده با روش پیش‌بینی

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC, LinearSVC
from sklearn.decomposition import PCA, NMF, TruncatedSVD
from sklearn.manifold import Isomap
%matplotlib inline
pipe = Pipeline([
('reduce_dim', PCA()),
('classify', SVC())
])
```

در کد زیر توجه داشته باشید که isomap از ماژول manifold ناشی می‌شود و الگوریتم فاکتوراسیون ماتریس غیر منفی (NMF) با استفاده از SVDها یک ماتریس را به عوامل غیر منفی تقسیم می‌کند، هدف اصلی آن در این بخش مقایسه عملکرد آن با الگوریتم‌های دیگر است، در پردازش زبان (NLP) که فاکتورهای ماتریس نمی‌توانند منفی باشند مفید است.

```
param_grid = [
{
'reduce_dim': [PCA(), NMF(),Isomap(),TruncatedSVD()],
'reduce_dim__n_components': [2, 3],
'classify' : [SVC(), LinearSVC()],
'classify__C': [1, 10, 100, 1000]
},
]
```

این پارامتر به شما امکان می‌دهد تا از طریق چند روش کاهش بعد همراه با دو نوع SVC: SVM خطی و SVC طبقه‌بندی آن را انجام دهید. حال جستجوی grid را اعمال می‌کنیم

```
grid = GridSearchCV(pipe, cv=3, n_jobs= 1 , param_grid=param_grid)
iris = load_iris()
grid.fit(iris.data, iris.target)
```

اکنون به بهترین پارامترها نگاه کنید تا بهترین مدل تعیین شود. PCA با SVC بهترین مدل می‌باشد

```
grid.best_params_
{'classify': SVC(C=10, cache_size=200, class_weight=None,
coef0=0.0,
decision_function_shape=None, degree=3, gamma='auto',
kernel='rbf',
max_iter=-1, probability=False, random_state=None,
shrinking=True,
tol=0.001, verbose=False),
'classify__C': 10,
'reduce_dim': PCA(copy=True, iterated_power='auto',
n_components=3, random_state=None,
svd_solver='auto', tol=0.0, whiten=False),
'reduce_dim__n_components': 3}
grid.best_score_
output: 0.98
```

جستجوی شبکه از cross-validation برای تعیین بهترین امتیاز استفاده می‌کند. در این حالت، تمام داده‌ها برای اعتبارسنجی three-fold استفاده می‌شود. نکته آخر در مورد خط لوله: روش‌های decomposition برای اولین مرحله کاهش ابعاد در داخل خط لوله کار می‌کنند، اما همه روش‌های منیفولد برای خطوط لوله طراحی نشده‌اند.